

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	KEERTHANA S	Reg. No.:	192471047
Date:	13.08.2026	Duration:	60 minutes

Code of Conduct

I KEERTHANA S (192471047) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Keerthana S

Annexure A – Code Review & Repository Evaluation

Problem Statement : Smart AI-Based Urban Street Lighting Management System

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
 - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
- Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

	organized.			
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Overview

1.1 Problem Statement

Traditional street lighting systems operate on fixed ON/OFF schedules regardless of real conditions, wasting electricity and delaying fault detection. Manual monitoring cannot reliably keep pace with a large, distributed network of streetlights, so malfunctioning units often go unnoticed for extended periods, compromising public safety. This project - the Smart AI-Based Urban Street Lighting Management System - was built to address this by combining IoT telemetry, an AI/ML engine, and a centralized dashboard so lighting can be monitored, optimized, and maintained proactively rather than reactively.

1.2 Summary of the Implemented Solution

The implemented solution is a containerized, multi-service application consisting of a Node.js/Express backend API, a Python/FastAPI AI microservice, a React monitoring dashboard, a PostgreSQL database, and an MQTT broker for IoT communication. Streetlight controllers publish periodic status telemetry over MQTT; the backend persists this data and forwards relevant signals to the AI engine, which classifies faults and computes adaptive brightness recommendations. Operators view live status, faults, and energy reports through the dashboard, and citizens can additionally report issues through a companion interface.

1.3 Project Objectives

- Continuously monitor the operational status of every streetlight in real time.
- Automatically detect faulty or malfunctioning streetlights without manual inspection.
- Optimize lighting intensity using AI based on traffic, pedestrian, and weather data.
- Reduce overall electricity consumption across the network.
- Predict maintenance requirements before failures occur.
- Generate energy usage and performance reports for administrators.
- Improve public safety through reliable, adaptive lighting.
- Support the municipality's smart-city initiatives with centralized, data-driven control.

1.4 Key Functionalities Delivered

Module	Key Functionality
backend (API)	Ingests IoT status data, exposes REST endpoints for lights/faults/users, triggers alert notifications.
ai-engine	Classifies faults (no output, flickering), computes adaptive brightness, predicts maintenance windows.
frontend (dashboard)	Displays live map of streetlight status, fault list, energy reports, and admin controls.
mqtt-broker	Receives real-time telemetry published by IoT streetlight

Module	Key Functionality
	controllers.
database	Persists streetlight records, fault history, energy logs, and user accounts.

2. Repository Organization

The repository follows a microservice-oriented layout, with each deployable component isolated into its own top-level directory.

```
streetlight-management-system/
├── backend/                                # Node.js/Express REST API
│   └── src/{controllers,models,routes,services,middleware,config}/
│       ├── tests/
│       ├── package.json, Dockerfile, .env.example
│   └── ai-engine/                          # Python AI/ML microservice
│       ├── models/, src/{fault_detection.py, adaptive_brightness.py,
│       ...}
│       ├── requirements.txt, Dockerfile
├── frontend/                              # React.js monitoring dashboard
│   ├── src/{components,pages,services}/
│   ├── package.json, Dockerfile
├── mqtt-broker/mosquitto.conf
├── database/init.sql
├── docs/{architecture-diagram.png, workflow-diagram.png}
├── screenshots/
├── .gitignore, .env.example, docker-compose.yml, README.md,
LICENSE
```

2.1 Structure Evaluation

Aspect	Observation	Rating
Separation of concerns	Each service (backend, ai-engine, frontend) is fully isolated with its own dependencies, Dockerfile, and source tree - no cross-service imports.	Strong
Internal layering	backend/src follows a standard MVC-like split (controllers/models/routes/services/middleware), which is immediately familiar to Node.js developers.	Strong
Naming conventions	Folders use lowercase-kebab or lowercase names (ai-engine, mqtt-broker); files use camelCase for JS (faultController.js) and snake_case for Python (fault_detection.py), matching each language's	Strong

Aspect	Observation	Rating
	idioms.	
Test placement	tests/ exists under backend/ but the AI engine and frontend do not yet have an equivalent dedicated test directory.	Needs improvement
Config/secrets handling	.env.example is present at the root and per-service, but there is no centralized documentation of which variables apply to which service.	Needs improvement
Documentation assets	docs/ holds architecture and workflow diagrams, keeping visual documentation separate from code.	Strong

2.2 How the Layout Supports Maintainability and Collaboration

- Independent service folders let backend, AI, and frontend contributors work without stepping on each other's files, reducing merge conflicts.
- Each service can be built, tested, and deployed independently via its own Dockerfile, which maps directly onto the Git branching strategy (feature branches scoped per service).
- A consistent internal structure (controllers/models/routes) means a new contributor can find relevant code quickly without a guided walkthrough.
- Keeping docs/ and screenshots/ outside the source folders prevents documentation churn from polluting code-only commit diffs.

3. Code Quality Review

Representative files from each service were reviewed for readability, modularity, adherence to coding standards, and inline documentation. Examples of both strong practice and areas needing improvement are shown below.

3.1 Example of Strong Practice - backend/src/controllers/faultController.js

```
/**
 * Logs a newly detected streetlight fault and triggers a
 * technician alert.
 * @route POST /api/faults
 */
async function logFault(req, res, next) {
  try {
    const { lightId, faultType, detectedAt } = req.body;

    if (!lightId || !faultType) {
      return res
        .status(400)
        .json({ error: "lightId and faultType are required" });
    }
  }
}
```

```

    const fault = await faultService.createFault({
      lightId,
      faultType,
      detectedAt,
    });
    await alertService.notifyTechnician(fault);

    return res.status(201).json(fault);
  } catch (err) {
    next(err); // delegated to centralized error-handling
  }
}

module.exports = { logFault };

```

Strengths:

- A JSDoc comment states the function's purpose and the route it serves.
- Input validation is performed before any business logic executes.
- Business logic is delegated to `faultService` and `alertService` rather than being inlined, keeping the controller thin and testable.
- Errors are passed to `next(err)` instead of being swallowed, relying on centralized middleware for consistent error responses.

3.2 Example Needing Improvement - `ai-engine/src/fault_detection.py`

```

def check(d):
    if d["lux"] == 0 and d["state"] == "ON":
        f = "No Output"
    elif d["lux"] > 0 and d["lux"] < 5 and d["flicker_count"] >
3:
        f = "Flickering"
    elif d["last_seen"] > 120:
        f = "Comm Loss"
    else:
        f = None
    if f is not None:
        db.insert("faults", {"light_id": d["id"], "type": f})
        requests.post(
            "http://backend:5000/api/alerts",
            json={"id": d["id"], "type": f}
        )
    return f

```

Issues Identified:

- Function and parameter names (check, d, f) are not descriptive, hurting readability.
- No docstring or type hints, despite this being a core business-logic function.
- Magic numbers (5, 3, 120) are hardcoded with no named constants or configuration, making thresholds hard to tune or discover.
- The function mixes three responsibilities - fault classification, database writes, and HTTP calls - violating single-responsibility and making unit testing difficult (it cannot be tested without a live database and network).
- No error handling around the requests.post call; a network failure here would raise an unhandled exception.

Suggested Refactor

```
NO_OUTPUT_LUX_THRESHOLD = 0
FLICKER_LUX_MAX = 5
FLICKER_COUNT_THRESHOLD = 3
COMM_LOSS_SECONDS = 120

def classify_fault(reading: dict) -> str | None:
    """Classify a telemetry reading into a fault type, or None if healthy."""
    if reading["lux"] == NO_OUTPUT_LUX_THRESHOLD and reading["state"] == "ON":
        return "No Output"
    lux, flicker = reading["lux"], reading["flicker_count"]
    if 0 < lux < FLICKER_LUX_MAX and flicker > FLICKER_COUNT_THRESHOLD:
        return "Flickering"
    if reading["last_seen"] > COMM_LOSS_SECONDS:
        return "Comm Loss"
    return None

# Persistence and alerting are handled by separate, testable functions
# that call classify_fault() rather than being embedded inside it.
```

Separating classification from persistence and alerting allows `classify_fault()` to be unit tested with plain dictionaries, with no database or network dependency.

3.3 Example - frontend/src/components/StatusMap.jsx

```
export default function StatusMap({ lights }) {
  return (
    <MapContainer center={[13.08, 80.27]} zoom={13}>
      <TileLayer
url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png" />
      {lights.map(light => (
        <Marker
          key={light.id}
          position={[light.lat, light.lng]}
          icon={getIcon(light.status)}
        >
          <Popup>{light.id} - {light.status}</Popup>
        </Marker>
      ))}
    </MapContainer>
  );
}
```

Observations:

- Component is small, purely presentational, and receives data via props - good separation from data-fetching logic.
- Using `key={light.id}` on the mapped Marker list follows React best practice and avoids re-render bugs.
- Minor improvement: `getIcon(light.status)` is imported from elsewhere but not memoized; for a large number of markers this could be wrapped in `useMemo` to avoid recomputation on every render.

3.4 Overall Code Quality Summary

Criterion	Backend (Node.js)	AI Engine (Python)	Frontend (React)
Readability	Good - descriptive names, consistent style	Mixed - some functions terse and under-documented	Good - small, focused components
Modularity	Strong - controller/service/model layering	Weak in places - logic, I/O, and persistence mixed	Strong - presentational vs. container components separated
Documentation	JSDoc present on public functions	Sparse - missing docstrings on core logic	Prop usage mostly self-explanatory but no PropTypes/TS
Standards compliance	ESLint + Prettier configured and passing	No linter (flake8/black)	ESLint configured via Create React

Criterion	Backend (Node.js)	AI Engine (Python)	Frontend (React)
		currently configured	App defaults

4. Version Control Evaluation

4.1 Commit History

```
* 9f3a1c2 (HEAD -> develop) fix: resolve merge conflict in docker-
compose.yml
* 7b2e441 merge: integrate fault-detection feature into develop
|\
| * 4d8f0aa test(backend): add unit tests for fault logging
endpoint
| * 1a6c2e9 feat(backend): add endpoint to log detected faults
| * e3f9b71 feat(ai-engine): implement rule-based fault
classification
|/
* c0a4d55 feat(frontend): add real-time status map component
* 92b1f3d feat(ai-engine): add adaptive brightness control
service
* 5e7a0c8 feat(backend): add streetlight status ingestion API
* 3f1d9b6 chore: configure Docker Compose for backend and
database
* d0e2a41 chore: initial commit - project scaffolding
* (main) 0a1b2c3 release: v1.0.0 - initial production release
```

4.2 Branching Strategy Analysis

Branch	Purpose	Evaluation
main	Stable, production-ready code; only release/hotfix branches merge here.	Correctly protected - no direct commits observed on main.
develop	Integration branch for completed features.	Used consistently as the merge target for all feature branches.
feature/*	Isolated work per feature, e.g. feature/fault-detection.	Naming convention is consistent and descriptive across the history.
release/*	Stabilization before merging into main.	Present for v1.0.0; recommend tagging releases (git tag) for easier rollback reference.
hotfix/*	Urgent production patches.	Not yet exercised in this history; process is defined but unproven under a real incident.

4.3 Commit Message Quality

- Conventional Commits prefixes (feat, fix, test, chore, release) are applied consistently, making the change type clear without opening the diff.
- Scopes in parentheses, e.g. feat(ai-engine), immediately identify which service was touched - useful in a multi-service monorepo.
- Commits are atomic: each one bundles a single logical change (one feature, one test addition, one fix) rather than mixing unrelated edits.
- One observed gap: a small number of early commits (chore: configure Docker Compose for backend and database) could be split further to isolate the backend and database configuration changes separately for finer-grained history.

4.4 Repository Maintenance Practices

Practice	Observed?	Comment
.gitignore coverage	Yes	node_modules/, __pycache__/, .env, and build/artifacts are excluded from tracking.
Branch cleanup	Yes	Merged feature branches are deleted locally and remotely after integration.
Protected main branch / PR reviews	Not evidenced	No branch protection rules or PR review records were found; recommended for future iterations.
Release tagging	Partial	A release/1.0 branch exists but no corresponding git tag (v1.0.0) was found in the history.
CI on push/PR	Not evidenced	No automated pipeline currently runs tests or linting on commits/PRs.

4.5 How Version Control Supports Collaborative Development

- Feature branches let backend, AI, and frontend contributors work simultaneously without blocking one another, merging cleanly through develop.
- The full history provides an audit trail for changes to safety-relevant logic (e.g., fault thresholds), which matters for a public-infrastructure system.
- Descriptive, conventional commit messages make it possible to generate a changelog automatically and to locate regressions quickly via git bisect.
- A shared remote (GitHub) with tracked branches keeps every contributor's work backed up and visible, rather than living only on individual machines.

5. Code Testing and Validation

The application was validated through a combination of automated unit tests and manual end-to-end testing of the running Docker Compose stack.

5.1 Automated Unit Tests

```
# backend/tests/fault.test.js
describe("POST /api/faults", () => {
  it("creates a fault record and returns 201 for valid input",
  async () => {
    const res = await request(app)
      .post("/api/faults")
      .send({ lightId: 1024, faultType: "No Output",
        detectedAt: "2026-08-13T10:15:32Z" });
    expect(res.statusCode).toBe(201);
    expect(res.body.lightId).toBe(1024);
  });

  it("returns 400 when lightId is missing", async () => {
    const res = await request(app)
      .post("/api/faults")
      .send({ faultType: "No Output" });
    expect(res.statusCode).toBe(400);
  });
});

$ npm test
PASS tests/fault.test.js
  POST /api/faults
    ✓ creates a fault record and returns 201 for valid input (42 ms)
    ✓ returns 400 when lightId is missing (11 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
```

5.2 Manual End-to-End Test Cases

ID	Test Case	Steps	Expected Result	Status
TC-01	Health check	curl http://localhost:5000/health	HTTP 200 with { "status": "ok" }	Pass
TC-	IoT status ingestion	Publish MQTT message to	Backend stores	Pass

ID	Test Case	Steps	Expected Result	Status
02		streetlight/1024/status	updated status within 30s	
TC-03	Fault detection	Publish reading with lux=0, state=ON	Fault logged as 'No Output' within 2 min	Pass
TC-04	Technician alert	Trigger TC-03	SMS/app notification sent within 1 min	Pass
TC-05	Dashboard live update	Trigger TC-03, open dashboard	Map marker for light 1024 turns red	Pass
TC-06	Invalid input handling	POST /api/faults with missing lightId	HTTP 400 with error message	Pass
TC-07	AI brightness adjustment	Simulate low traffic on a street segment	Brightness reduced within safety threshold	Pass
TC-08	Citizen fault report	Submit report via mobile app	New complaint ticket created within 1 min	Pass

5.3 Evidence of Successful Execution

Supporting screenshots (to be inserted in the final submission) include: (1) the terminal output of npm test showing all suites passing, (2) docker compose ps confirming all containers healthy during testing, (3) the browser dashboard showing a live fault marker after TC-03/TC-05, and (4) the technician's received SMS/app alert from TC-04.

5.4 Testing Gaps Identified

- The ai-engine service currently has no automated test suite (e.g., pytest) covering fault classification or brightness logic - all validation there was manual.
- No integration tests exist that spin up the full Docker Compose stack automatically in CI; end-to-end testing was performed manually for this report.
- Frontend components do not yet have unit tests (e.g., React Testing Library) verifying rendering behavior for different fault states.

6. Repository Documentation

6.1 Current README.md Contents

```
# Smart AI-Based Urban Street Lighting Management System
```

AI-powered, IoT-enabled platform for monitoring, optimizing, and maintaining municipal street lighting.

Getting Started

1. Clone the repository
2. Copy .env.example to .env in each service folder
3. Run: docker compose up -d
4. Open <http://localhost:3000> for the dashboard

Services

- backend (port 5000)
- ai-engine (port 8000)
- frontend (port 3000)
- database (PostgreSQL, port 5432)
- mqtt-broker (port 1883)

6.2 Documentation Completeness Evaluation

Element	Present?	Assessment
Project description	Yes	Clear one-line summary of purpose, but lacks a longer overview of architecture or objectives.
Installation instructions	Yes	Basic clone-and-run steps are present and accurate for Docker-based setup.
Usage guide	Partial	Lists service URLs but does not explain key dashboard features, sample API calls, or how to publish test MQTT messages.
Environment variables reference	Partial	.env.example files exist per service, but README does not summarize which variables are required vs. optional.
Dependencies list	Partial	Implied through package.json/requirements.txt but not summarized in the README itself.
Architecture diagram	Yes (external file)	Stored under docs/, but not embedded or linked directly from the README.
Contribution guidelines	No	No CONTRIBUTING.md or branching-strategy explanation is provided for new contributors.
License	Yes	LICENSE file present at the repository root.
Testing instructions	No	README does not explain how to run npm test or

Element	Present?	Assessment
		a Python test suite.

6.3 Suggested Improvements

- Embed or link the architecture diagram directly in the README so new contributors see the system layout immediately.
- Add a 'Running Tests' section documenting npm test for the backend and the recommended pytest setup for the AI engine.
- Add a table of required environment variables per service, with descriptions and example values, consolidated in the top-level README.
- Add a CONTRIBUTING.md describing the Gitflow branching strategy, commit message convention, and PR review expectations.
- Include a short 'API Reference' section or link to a generated OpenAPI/Swagger spec for the backend endpoints.
- Add badges (build status, license, coverage) once CI is introduced, to give an at-a-glance repository health summary.

7. Code Review Findings and Recommendations

7.1 Summary of Findings

Area	Overall Rating	Key Finding
Repository organization	Strong	Clear microservice separation and consistent internal layering support collaboration well.
Code quality	Good, with gaps	Backend is clean and modular; the AI engine mixes responsibilities and lacks documentation/typing.
Version control	Good	Gitflow is applied consistently with clear commit messages; branch protection and release tagging are missing.
Testing	Needs improvement	Backend has unit tests and passing manual E2E cases; AI engine and frontend lack automated tests.
Documentation	Needs improvement	README covers basic setup but omits testing instructions, env-variable reference, and contribution guidelines.

7.2 Recommendations

Code Quality

- Refactor `ai-engine/src/fault_detection.py` to separate classification, persistence, and alerting into distinct, independently testable functions (as demonstrated in Section 3.2).
- Introduce `flake8/black` for Python and enforce `ESLint/Prettier` consistently across all services via a pre-commit hook.
- Add `docstrings/JSDoc` to all public functions, especially safety-relevant logic such as fault classification and brightness thresholds.

Repository Management

- Enable branch protection on `main` and `develop`, requiring at least one PR review before merge.
- Tag releases with semantic version tags (e.g., `v1.0.0`) alongside the `release/*` branch for easy rollback reference.
- Add a dedicated `tests/` directory convention for the AI engine and frontend to mirror the backend's structure.

Documentation

- Expand the `README` with a testing guide, environment-variable reference table, and an embedded architecture diagram.
- Add a `CONTRIBUTING.md` documenting the Gitflow workflow and commit message convention used in this project.

Testing

- Add a `pytest` suite for the AI engine covering `classify_fault()` and the adaptive brightness logic with edge-case inputs.
- Add `React Testing Library` tests for key components such as `StatusMap` and the fault list.
- Introduce a CI pipeline (e.g., `GitHub Actions`) to run all test suites and linters automatically on every push and pull request.

Future Enhancements

- Add integration tests that boot the full `Docker Compose` stack in CI and verify the IoT-to-dashboard data flow end to end.
- Introduce a secrets manager for production credentials instead of plain `.env` files.
- Add code coverage reporting to track testing progress over time as new features are added.

Overall, the repository demonstrates a solid architectural foundation and disciplined use of Git for a multi-service project. The primary opportunities for improvement lie in extending automated testing and documentation to match the quality already present in the backend service, and in formalizing repository governance (branch protection, release tagging, CI) as the project matures toward production use.